Campus Sports DBMS — Complete Design & Architectural Handout

> **Audience**: Complete Ground-Zero Guide for Lab Defense, Viva Examination, and Deep Technical Understanding.

> **Database Engine**: MySQL 8.0 Community Edition (InnoDB, `utf8mb4_0900_ai_ci`).

> **Schema Size**: 18 Normalized Relations, 6 Active Triggers, 3 Analytical Views.

Table of Contents

1. Core Philosophy & Architecture: Starting From Zero
2. Fundamental Relational Concepts Explained
3. The 5 Logical Clusters of the 18 Tables
4. Deep-Dive: How Every Relation Works and Connects
5. Key Architectural Patterns (Viva Goldmines)
6. Database Triggers: What They Are & Deep Analysis
7. The Standings Engine: Why We Use Views Instead of Stored Tables
8. Viva & Oral Defense Quick Reference

---

1. Core Philosophy & Architecture: Starting From Zero

What is an RDBMS?

A **Relational Database Management System (RDBMS)** organizes data into structured two-dimensional grids called **tables** (or **relations**), consisting of columns (attributes) and rows (tuples). Relationships between tables are created using values stored in columns, rather than physical pointers.

The Philosophy of This Project

In many novice web applications, database safety is treated as an afterthought: developers write code in JavaScript or Python to validate rules (e.g. checking if two matches clash or if a student is old enough). If someone connects via MySQL Workbench, imports a CSV, or has a bug in their backend code, the database gets corrupted.

In our project:

> **The Database is the Engine; the Node.js/Express Web App is only a Thin Lens.**

Every business rule, scheduling limit, eligibility check, and data consistency rule is enforced **at the database tier** using:

- Strict **Primary and Candidate Unique Keys**
- **Foreign Key Referential Actions** (`CASCADE`, `RESTRICT`, `SET NULL`)
- Database **CHECK Constraints**
- Event-driven **Database Triggers**

- Virtual derived **SQL Views**

Even if the web server crashes or a rogue query is executed directly in MySQL, **the database will refuse to enter an invalid state**.

---

2. Fundamental Relational Concepts Explained

Before analyzing the 18 tables, understand these three building blocks:

A. Keys

- **Primary Key (PK)**: A column (or set of columns) that uniquely identifies each individual row in a table. In our schema, every strong entity uses a surrogate integer `<table>_id` with `AUTO_INCREMENT`.
- **Candidate / Unique Key (UQ)**: Other columns that must be unique across the table (e.g., `student.roll_number`, `department.name`, `(tournament_id, name)`).

B. Foreign Keys & Referential Actions

A **Foreign Key (FK)** is a column in a child table pointing to the Primary Key of a parent table. It prevents orphan rows. When a parent row is modified or deleted, MySQL enforces a **Referential Action**:

1. `ON DELETE CASCADE`:

If the parent is deleted, delete all children automatically.

Example: If an `event` is deleted, its `stage` records and `eligibility_rule` are deleted automatically because they have no meaning without that event.

1. `ON DELETE RESTRICT`:

Rejects the deletion of the parent if any child rows still reference it.

Example: You cannot delete a `department` if there are still students or teams enrolled in it. You cannot delete a `team` if it has match history.

1. `ON DELETE SET NULL`:

Leaves the child row intact, but clears the foreign key column to `NULL`.

Example: If a student who was a team captain leaves college (`student` deleted), `team.captain_id` is set to `NULL` so the team itself is not erased.

C. Database `CHECK` Constraints

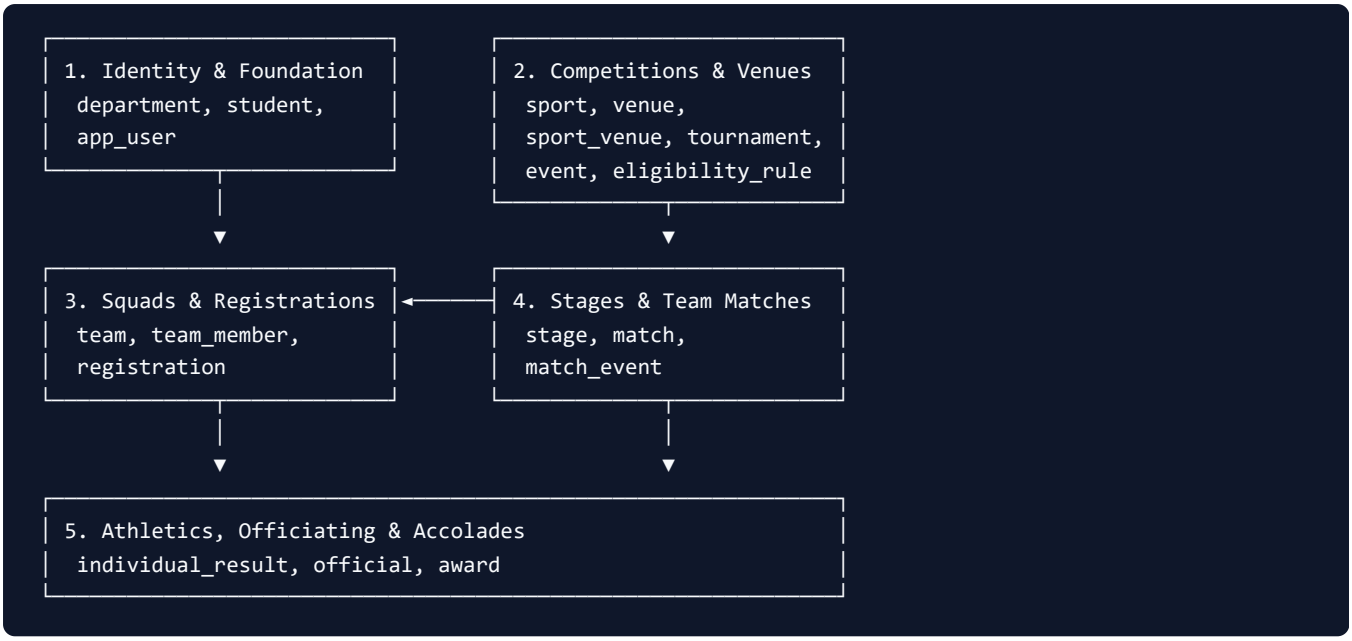
In MySQL 8.0, `CHECK` constraints validate boolean expressions before allowing an `INSERT` or `UPDATE`. If the condition evaluates to `FALSE`, the database aborts the transaction with an error.

Example: `CONSTRAINT ck_tournament_dates CHECK (end_date >= start_date)`.

---

3. The 5 Logical Clusters of the 18 Tables

To understand 18 tables effortlessly, divide them into **5 logical functional clusters**:



1. **Cluster 1: Identity & Foundation** (`department` , `student` , `app_user`): Campus academic units, students, and authentication identities.
2. **Cluster 2: Competitions & Venues** (`sport` , `venue` , `sport_venue` , `tournament` , `event` , `eligibility_rule`): What games are played, where they can be played, tournament editions, specific events, and age/year criteria.
3. **Cluster 3: Squads & Registrations** (`team` , `team_member` , `registration`): Teams entered by departments, athlete rosters with jersey numbers, and formal entry records into events.
4. **Cluster 4: Stages & Team Matches** (`stage` , `match` , `match_event`): Group/knockout rounds, head-to-head fixtures with scores/times/venues, and minute-by-minute goals/cards.
5. **Cluster 5: Athletics, Officiating & Accolades** (`individual_result` , `official` , `award`): Individual track/field results, referees/umpires assigned to matches or heats, and medals/trophies.

---

4. Deep-Dive: How Every Relation Works and Connects

Cluster 1: Identity & Foundation

1. `department`

- **Real-World Meaning**: Represents an academic branch (e.g. Computer Science, Mechanical) or residential hall/hostel competing in the sports fest.
- **Primary Key**: `department_id`
- **Candidate Key**: `name` (Unique)
- **Columns**: `name` , `department_type` (`'department'` , `'hostel'`)
- **Relationships**:
- **1 : N with** `student` : One department has many students.
- **1 : N with** `team` : One department enters many sports teams.
- **Integrity Action**: `ON DELETE RESTRICT` prevents wiping out an academic department if students or teams are active.

2. `student`

- **Real-World Meaning**: A verified college student eligible to compete, join teams, or captain squads.

- **Primary Key**: `student_id`
- **Foreign Keys**: `department_id` → `department.department_id` (`RESTRICT` delete, `CASCADE` update).
- **Candidate Key**: `roll_number` (Unique college identifier).
- **CHECK Constraint**: `academic_year BETWEEN 1 AND 6`.
- **Relationships**:
- Can captain a `team` (0..1 : 1).
- Can belong to multiple teams via `team_member` (1 : N).
- Can register directly for individual athletics events via `registration` (1 : N).
- Can have one login user account in `app_user` (1 : 0..1).

3. `app_user`

- **Real-World Meaning**: Authenticated user accounts for the web system.
- **Primary Key**: `app_user_id`
- **Foreign Key**: `student_id` → `student.student_id` (Nullable, Unique).
- **Role ENUM**: `'admin'`, `'coordinator'`, `'player'`, `'viewer'`.
- **Design Note**: A viewer, admin, or coordinator does not require a student record (`student_id` can be `NULL`). If a student logs in, their account links 1-to-1 with their student identity.

Cluster 2: Competitions & Facilities

4. `sport`

- **Real-World Meaning**: The sport discipline (e.g. Football, Basketball, 100 Metres Sprint).
- **Primary Key**: `sport_id`
- **Key Attribute**: `is_team_sport` (`BOOLEAN`).
- If `TRUE` : Competition follows team squads, groups, home/away fixtures (`match`), and points tables.
- If `FALSE` : Competition follows individual athletes, heats, and ranked metric results (`individual_result`).

5. `venue`

- **Real-World Meaning**: A physical ground, court, synthetic track, or indoor hall on campus.
- **Primary Key**: `venue_id`
- **Candidate Key**: `name` (e.g. "Main Stadium Turf", "Court B").
- **Relationships**: Hosts scheduled matches and athletics heats. Protected against time conflicts by database triggers.

6. `sport_venue` **(Many-to-Many Associative Entity)**

- **Real-World Meaning**: Specifies which sports are physically compatible with which campus venues.
- **Composite Primary Key**: `(sport_id, venue_id)`
- **Foreign Keys**:
- `sport_id` → `sport.sport_id` (`CASCADE`)
- `venue_id` → `venue.venue_id` (`CASCADE`)
- **Why it exists**: A basketball court cannot host a 100m sprint track event, nor can an indoor hall host football. This relation enforces facility suitability.

7. `tournament`

- **Real-World Meaning**: The overall tournament edition (e.g., "Campus Sports Meet 2026").

- **Primary Key**: `tournament_id`
- **Candidate Key**: `(name, start_date)`
- **CHECK Constraint**: `ck_tournament_dates CHECK (end_date >= start_date)`. Ensures chronological sanity.
- **Relationships**: Parent container of all events.

8. `event`
- **Real-World Meaning**: A specific competition category under a tournament for a given sport (e.g. "Men's 100m Sprint" or "Inter-Department Football Championship").
- **Primary Key**: `event_id`
- **Foreign Keys**:
- `tournament_id` → `tournament.tournament_id` (`RESTRICT`)
- `sport_id` → `sport.sport_id` (`RESTRICT`)
- **Attributes**: `gender_category` (`'men'` , `'women'` , `'mixed'` , `'open'`), `format` (`'group_knockout'` , `'knockout'` , `'heats'`).
- **Candidate Key**: `(tournament_id, name)` (Cannot have duplicate event names in the same tournament).

9. `eligibility_rule` **(Weak / Auxiliary Entity)**
- **Real-World Meaning**: Quantitative criteria governing who is allowed to enter an event.
- **Primary Key**: `eligibility_rule_id`
- **Foreign Key**: `event_id` → `event.event_id` (`CASCADE` , Unique). Exactly one rule per event.
- **CHECK Constraints**: Ensures `max_year >= min_year` and `max_age >= min_age` .
- **Attributes**: `min_year` , `max_year` , `gender` , `min_age` , `max_age` .
- **How it works**: When a student attempts to register for an event, triggers read this table to approve or abort the transaction.

Cluster 3: Squads & Registrations

10. `team`
- **Real-World Meaning**: A department-sponsored team entered to compete in team sports (e.g., "CS Strikers", "Mech Warriors").
- **Primary Key**: `team_id`
- **Foreign Keys**:
- `department_id` → `department.department_id` (`RESTRICT` delete, `CASCADE` update).
- `captain_id` → `student.student_id` (`SET NULL` delete).
- **Candidate Key**: `(department_id, name)` (A department cannot have two teams with the identical name).

11. `team_member` **(Many-to-Many Associative Entity)**
- **Real-World Meaning**: Roster connecting students to teams with jersey numbers.
- **Composite Primary Key**: `(team_id, student_id)`
- **Foreign Keys**:
- `team_id` → `team.team_id` (`CASCADE`)
- `student_id` → `student.student_id` (`CASCADE`)
- **Candidate Key**: `(team_id, jersey_number)` (No two players on the same team can wear the same jersey number).

- **CHECK Constraint**: `jersey_number BETWEEN 0 AND 99`.

12. `registration` (The Gateway Relation)

- **Real-World Meaning**: The official entry of an entity into a tournament event.
- **Primary Key**: `registration_id`
- **Foreign Keys**:
- `event_id` → `event.event_id` (`RESTRICT`)
- `student_id` → `student.student_id` (Nullable, `RESTRICT`)
- `team_id` → `team.team_id` (Nullable, `RESTRICT`)
- **Exclusive CHECK**: `CONSTRAINT ck_registration_exactly_one_entrant CHECK ((student_id IS NOT NULL) <> (team_id IS NOT NULL))`
- **Candidate Keys**:
- `(event_id, student_id)`: A student cannot register for the same event twice.
- `(event_id, team_id)`: A team cannot register for the same event twice.

Cluster 4: Competition Stages & Team Fixtures

13. `stage`

- **Real-World Meaning**: A sequential competitive phase within an event (e.g. "Group A", "Quarter Finals", "Heats Round 1").
- **Primary Key**: `stage_id`
- **Foreign Key**: `event_id` → `event.event_id` (`CASCADE`).
- **Candidate Keys**:
- `(event_id, stage_order)`: Stage sequencing must be unique (e.g., 1, 2, 3).
- `(event_id, name)`: Stage names must be distinct within the event.
- **ENUM**: `stage_type` (`'group'`, `'quarter_final'`, `'semi_final'`, `'final'`, `'heat'`).

14. `match` (Reserved Identifier: Escaped with Backticks)

- **Real-World Meaning**: A scheduled head-to-head contest between two teams.
- **Primary Key**: `match_id`
- **Foreign Keys**:
- `stage_id` → `stage.stage_id` (`RESTRICT`)
- `venue_id` → `venue.venue_id` (`RESTRICT`)
- `home_team_id` → `team.team_id` (`RESTRICT`)
- `away_team_id` → `team.team_id` (`RESTRICT`)
- **CHECK Constraints**:
- `ck_match_distinct_teams CHECK (home_team_id <> away_team_id)`: A team cannot play against itself!
- `ck_match_duration CHECK (duration_minutes BETWEEN 1 AND 600)`.
- **State Management**: `status` (`'scheduled'`, `'live'`, `'completed'`, `'cancelled'`).

15. `match_event`

- **Real-World Meaning**: Granular incident log occurring during a match (e.g. goals, red cards, wickets).
- **Primary Key**: `match_event_id`
- **Foreign Keys**:

- `match_id` → `match.match_id` (`CASCADE`)
- `student_id` → `student.student_id` (Nullable player who scored/fouled, `RESTRICT`)
- `team_id` → `team.team_id` (Nullable team, `RESTRICT`)
- **Attributes**: `minute` , `event_type` (`'goal'` , `'wicket'` , `'yellow_card'` , `'red_card'` , etc.), `details` .

Cluster 5: Athletics, Officiating & Accolades

16. `individual_result`

- **Real-World Meaning**: Recorded time, distance, or score for a student competing in an individual athletics heat.
- **Primary Key**: `individual_result_id`
- **Foreign Keys**:
- `registration_id` → `registration.registration_id` (`CASCADE`)
- `stage_id` → `stage.stage_id` (`CASCADE`)
- `venue_id` → `venue.venue_id` (`RESTRICT`)
- **Candidate Key**: `(registration_id, stage_id)` : An entrant has exactly one result per heat stage.
- **Attributes**: `result_value` (e.g., 10.420), `result_unit` (`'seconds'` , `'metres'` , `'points'`), `result_rank` .

17. `official`

- **Real-World Meaning**: Referees, line judges, or timekeepers appointed to officiate matches or track heats.
- **Primary Key**: `official_id`
- **Foreign Keys**:
- `match_id` → `match.match_id` (Nullable, `RESTRICT`)
- `individual_result_id` → `individual_result.individual_result_id` (Nullable, `RESTRICT`)
- **Exclusive CHECK**: `CONSTRAINT ck_official_exactly_one_assignment CHECK ((match_id IS NOT NULL) <> (individual_result_id IS NOT NULL))` .

18. `award`

- **Real-World Meaning**: Podiums, trophies, and awards granted at the conclusion of an event.
- **Primary Key**: `award_id`
- **Foreign Keys**:
- `event_id` → `event.event_id` (`RESTRICT`)
- `student_id` → `student.student_id` (Nullable, `RESTRICT`)
- `team_id` → `team.team_id` (Nullable, `RESTRICT`)
- **Exclusive CHECK**: `CONSTRAINT ck_award_exactly_one_recipient CHECK ((student_id IS NOT NULL) <> (team_id IS NOT NULL))` .
- **Attributes**: `award_type` (`'gold'` , `'silver'` , `'bronze'` , `'player_of_tournament'` , `'department_trophy'`), `award_rank` , `awarded_at` .

---

5. Key Architectural Patterns (Viva Goldmines)

Examiners love asking about real-world relational design patterns. Here are the core patterns in this schema:

Pattern 1: The Exclusive ARC Pattern (Disjoint Polymorphism)

- **The Problem**: A registration can belong to a **Student** (for 100m sprint) OR a **Team** (for Football). In standard SQL, a foreign key can only reference one table.
- **Bad Solution (Anti-pattern)**: Storing `entrant_type` ('student'/'team') and an untyped integer `entrant_id`. This destroys referential integrity because MySQL cannot enforce foreign keys against dynamic tables.
- **Our Solution (The Relational Arc)**:

Store two nullable foreign keys: `student_id` and `team_id`, governed by a mathematical `CHECK`:

```
CONSTRAINT ck_registration_exactly_one_entrant
CHECK ((student_id IS NOT NULL) <> (team_id IS NOT NULL))
```

In boolean logic, `<>` (not equals) functions as an **Exclusive OR (XOR)**:

- Both NULL? `(FALSE) <> (FALSE)` → `FALSE` (Rejected!)
- Both filled? `(TRUE) <> (TRUE)` → `FALSE` (Rejected!)
- Exactly one filled? `(TRUE) <> (FALSE)` → `TRUE` (Accepted!)
- **Why `ON DELETE RESTRICT` is required**:

MySQL 8.0.46 prohibits cascading referential actions (`ON DELETE CASCADE` or `SET NULL`) on foreign keys participating in a `CHECK` expression. Therefore, all ARC foreign keys must use `ON DELETE RESTRICT ON UPDATE RESTRICT`.

Pattern 2: Deliberate Denormalization with Trigger Synchronization

- **The Problem**: Should match scores be calculated on the fly by counting `match_event` rows, or stored directly on `match`?
- **Our Design Decision**:
- `match` directly stores `home_score` and `away_score`.
- `match_event` stores every individual goal/wicket.
- **Why**: High-frequency dashboard queries, standings tables, and schedules require instant score access without running expensive joins on millions of game events.
- **How Integrity is Maintained**:

The database trigger `trg_update_score` listens for `AFTER INSERT ON match_event`. When a `'goal'` event is logged, the trigger increments `home_score` or `away_score` inside the same atomic database transaction.

Pattern 3: Dual Sport Progression (Team vs Individual)

- Our schema uses a single unified database for radically different sports:
- Team Sports follow: `team` → `registration` → `stage` → `match` → `match_event`.
- Athletics follow: `student` → `registration` → `stage` → `individual_result`.
- Both paths converge at `stage` and terminate at `award`.

---

6. Database Triggers: What They Are & Deep Analysis

What is a Database Trigger?

A **Trigger** is a procedural database object that is bound to a table and executed **automatically** by the MySQL engine whenever a specific Data Manipulation Language (DML) event (`INSERT` , `UPDATE` , or `DELETE`) occurs on that table.

Triggers have access to special virtual row variables:

- `NEW` : Represents the incoming data being inserted or updated.
- `OLD` : Represents the existing row data prior to update or deletion.

Why use Triggers instead of Express/Node.js Code?

Aspect	Application Code (Node.js)	Database Trigger (MySQL)
Bypassable?	Yes (Direct SQL, Workbench, script imports bypass it)	**Never** (Enforced inside the storage engine)
Transaction Safety	Requires manual `BEGIN...COMMIT` handling	**Automatic** (Runs inside the invoking ACID transaction)
Race Conditions	Prone to read-then-write concurrency bugs	**Protected** (Runs under InnoDB row/table locks)
Error Handling	Throws JavaScript errors	Uses standard `SIGNAL SQLSTATE '45000'`

The 6 Active Triggers in Campus Sports

1. `trg_check_eligibility_insert`

- **Timing & Table**: `BEFORE INSERT ON registration`
- **Purpose**: Prevents illegal student registrations based on demographic and academic criteria.
- **How It Works**:

1. Inspects `NEW.student_id` . If `NULL` (i.e. team registration), it bypasses student checks.
2. Queries `eligibility_rule` and `student` joining on `event_id` .
3. Computes the student's current age: `TIMESTAMPDIFF(YEAR, s.date_of_birth, CURRENT_DATE)` .
4. Checks 3 criteria:

- Does `student.gender` match `eligibility_rule.gender` ?
- Does `student.academic_year` fall between `min_year` and `max_year` ?
- Does `student.age` fall between `min_age` and `max_age` ?

1. If any rule is violated, it invokes:

`SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Student ... does not satisfy event eligibility'` .

2. `trg_check_eligibility_update`

- **Timing & Table**: `BEFORE UPDATE ON registration`
- **Purpose**: Catches attempts to swap the `student_id` or `event_id` of an existing approved registration to an ineligible entrant.

3. `trg_venue_clash_insert` (Temporal Conflict Resolution)

- **Timing & Table**: `BEFORE INSERT ON match`
- **Purpose**: Prevents double-booking any campus pitch, court, or hall.
- **The Mathematics of Time Interval Overlap**:

Two time intervals $[S_1, E_1]$ and $[S_2, E_2]$ overlap if and only if:

$$S_1 < E_2 \quad \text{AND} \quad E_1 > S_2$$

- **How It Works**:

When scheduling `NEW` at `NEW.venue_id` from `NEW.starts_at` for `NEW.duration_minutes`:

```
  WHERE existing_match.venue_id = NEW.venue_id
    AND existing_match.starts_at < DATE_ADD(NEW.starts_at, INTERVAL NEW.duration_minutes MINUTE)
    AND DATE_ADD(existing_match.starts_at, INTERVAL existing_match.duration_minutes MINUTE) > NEW.starts_a
```

If any row matches, it rejects the insert: `'Venue is already occupied during this match time'`.

4. `trg_venue_clash_update`

- **Timing & Table**: `BEFORE UPDATE ON match`
- **Purpose**: Checks for venue clashes when rescheduling a match or changing its venue/duration, while deliberately excluding itself (`existing_match.match_id <> OLD.match_id`).
- **Design Note**: If an update only modifies `home_score` or `away_score`, the trigger skips the clash check entirely to avoid unnecessary locks.

5. `trg_update_score`

- **Timing & Table**: `AFTER INSERT ON match_event`
- **Purpose**: Synchronizes match events with scorecards.
- **How It Works**:

1. Checks if `NEW.event_type = 'goal'`.
2. Confirms that `NEW.team_id` is actually one of the teams playing in `NEW.match_id` (either `home_team_id` or `away_team_id`). If not, it signals an error.
3. Executes an internal `UPDATE match SET home_score = home_score + 1` (or `away_score`).

6. `trg_lock_final_match` (Historical Immutability)

- **Timing & Table**: `BEFORE UPDATE ON match`
- **Purpose**: Enforces sports integrity by permanently locking finished matches.
- **How It Works**:

If `OLD.status = 'completed'`, any attempt to tamper with scores, teams, venues, stages, or dates raises:

`SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Completed matches cannot be edited'`.

Once a game is final, history cannot be rewritten without administrative audit intervention.

---

7. The Standings Engine: Why We Use Views Instead of Stored Tables

In our database, there is **no table called** `points_table`. Instead, standings are computed by a dynamic database view: `v_points_table`.

What is a View?

A **View** is a saved, named `SELECT` query stored in the database dictionary. It acts as a virtual table. When you run `SELECT * FROM v_points_table`, MySQL executes the underlying query against the base tables.

Why Standings Must Be a View:

1. **Zero Redundancy / No Update Anomalies**:

If team points were stored in a physical table, every score update would require updating `match`, `match_event`, and `points_table`. If a transaction failed midway, the points table would be out of sync with actual scores.

1. **Reproducible Truth**:

Standings are purely derived data. If you recalculate points tomorrow or fix a disputed score, the standings table is instantly 100% accurate.

How `v_points_table` Computes Points:

Under the hood, `v_points_table` aggregates match records:

- **Matches Played (P)**: Count of completed matches where the team was either home or away.
- **Wins (W)**: Count of matches where team was home and `home_score > away_score`, plus away wins.
- **Draws (D)**: Count of completed matches where `home_score = away_score`.
- **Losses (L)**: Count of matches where the team scored fewer points.
- **Goal Difference (GD)**:

$$\text{Signed Goals For} - \text{Signed Goals Against}$$

(Note: Cast to signed integers to prevent MySQL unsigned subtraction underflow!)

- **Total Points**:

$$\text{Points} = (W \times 3) + (D \times 1) + (L \times 0)$$

- **Position Ranking**:

Computed using the MySQL 8.0 window function:

```
DENSE_RANK() OVER (
    PARTITION BY m.stage_id
    ORDER BY points DESC, goal_difference DESC
) AS points_position
```

---

8. Viva & Oral Defense Quick Reference

Typical Examiner Question	Concise, Confident Answer
"What normal form is your database in?"	"The schema is in Boyce-Codd Normal Form (BCNF). Every relation is in 1NF (atomic attributes), 2NF (no partial functional dependencies on composite keys), and 3NF (no transitive dependencies). In all non-trivial functional dependencies $X \to Y$, X is a superkey. The only deliberate denormalization is storing aggregate match scores on `match`, which is strictly synchronized by our `trg_update_score` trigger."
"Why do you have `match_id` and `individual_result_id` both in `official`?"	"Officials can referee team matches or judge athletics heats. Since SQL does not permit a foreign key to point to multiple tables dynamically, we used the Relational Arc Pattern with two nullable foreign keys and a CHECK constraint: `(match_id IS NOT NULL) <> (individual_result_id IS NOT NULL)` to guarantee an official is assigned to exactly one target."
"What happens if someone deletes a department?"	"MySQL aborts the deletion with an error (`ON DELETE RESTRICT`). A department can only be deleted if it has no enrolled students and no competing teams."
"How do you prevent scheduling two matches on the same ground at the same time?"	"We enforce temporal non-overlap at the database tier using `trg_venue_clash_insert` and `trg_venue_clash_update`. The trigger computes $[starts_at, starts_at + duration]$ and signals error `45000` if another fixture occupies that venue during the interval."
"Why is `match` wrapped in backticks in your SQL?"	"`MATCH` is a reserved keyword in MySQL 8.0 (used for full-text search `MATCH...AGAINST`). To maintain singular snake_case naming standards without parser errors, we escape it as `` `match` ``."
"Where are standings stored?"	"Standings are not stored in any table. They are computed dynamically by the database view `v_points_table` using window functions like `DENSE_RANK()`. This eliminates update anomalies and guarantees that standings always match raw score history."
"What storage engine did you use and why?"	"We use InnoDB with `utf8mb4_0900_ai_ci`. InnoDB is essential because it supports ACID transactions, row-level locking, foreign key referential integrity constraints, and CHECK constraint enforcement."

Created for the Campus Sports Tournament DBMS Project. Keep this handout handy for oral viva reviews and code defense.